

1. Queue $\rightarrow$ get middle $\Theta(1)$

DSA - row 6 II

ADT

1) Implement Queue using a Priority Queue.

- a) - operations specs for Pr. Queue: init, push, pop, top, isEmpty
- b) - repres. of the queue
- c) - implem. all queue op.
- explain how you treat priorities of the elems.

relies on priority

ADT Priority Queue

b) Queue:

pg: PriorityQueue
nextPriority: Int

TElemPQ

elem: TElem
priority: TPriority

$\hookrightarrow$ remembers the priority for the next inserted element (force it to act like a normal queue)

PriorityQueue

cap: Int
nrElem: Int
elems: TElemPQ []
relation: TRelation

c) init

Subalgorithm init(q) is

```

init(q.pg)
q.nextPriority  $\leftarrow$  1

```

"highest priority"
= arrival time

Normal queue (FIFO)

only Subalgorithm enqueue(q, elem) is

```

push(q.pg, q.nextPriority)
q.nextPriority  $\leftarrow$  q.nextPriority + 1

```

$\rightarrow$ relies only on arrival time

function dequeue(q) is

```

if isEmpty(q.pg) then
    @throw...

```

```

dequeue  $\leftarrow$  top(q.pg);
pop(q.pg)

```

alg. $\leftarrow$ does not stop here!

function top(q) is

```

if isEmpty(q.pg) then
    @throw...

```

```

front  $\leftarrow$  top(q.pg);

```

function isEmpty(q) is

```

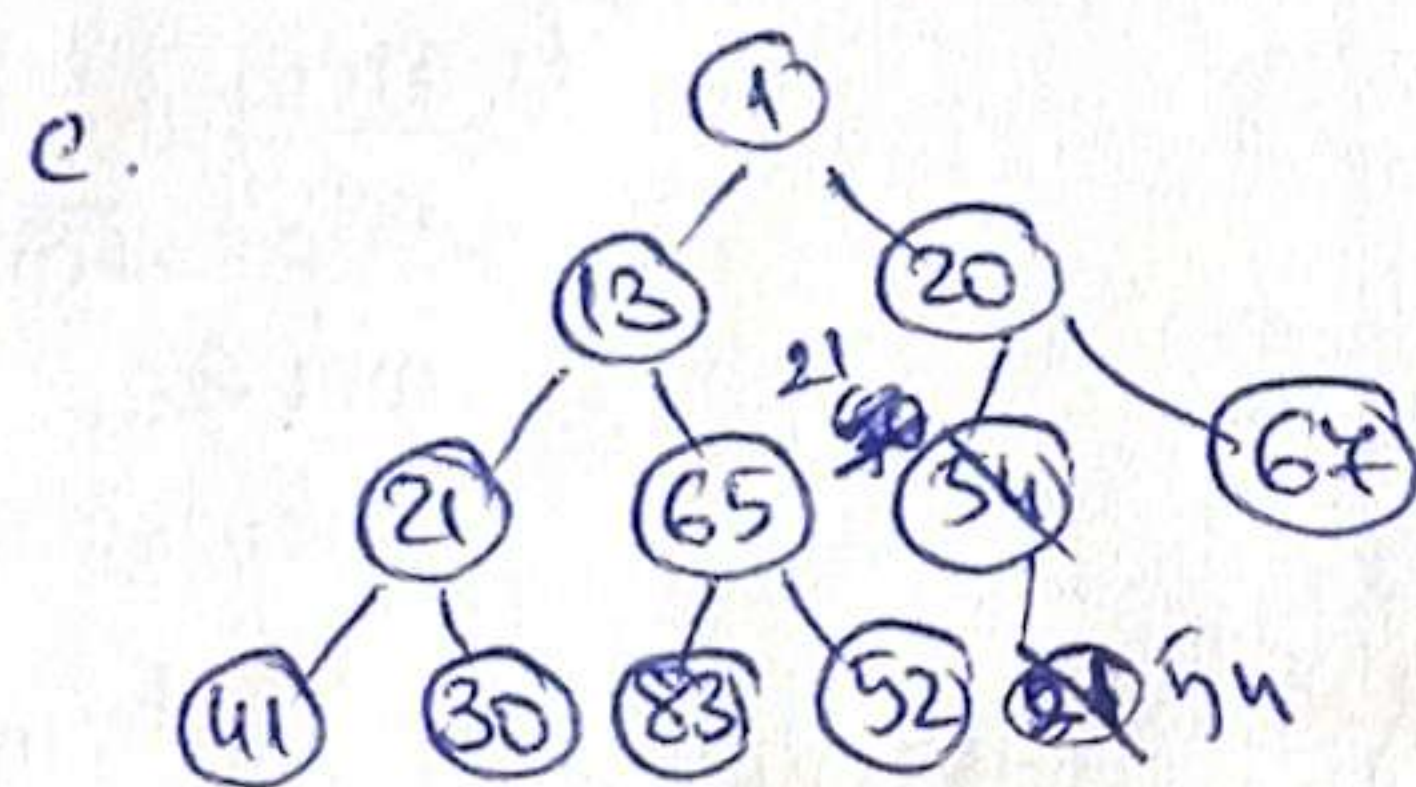
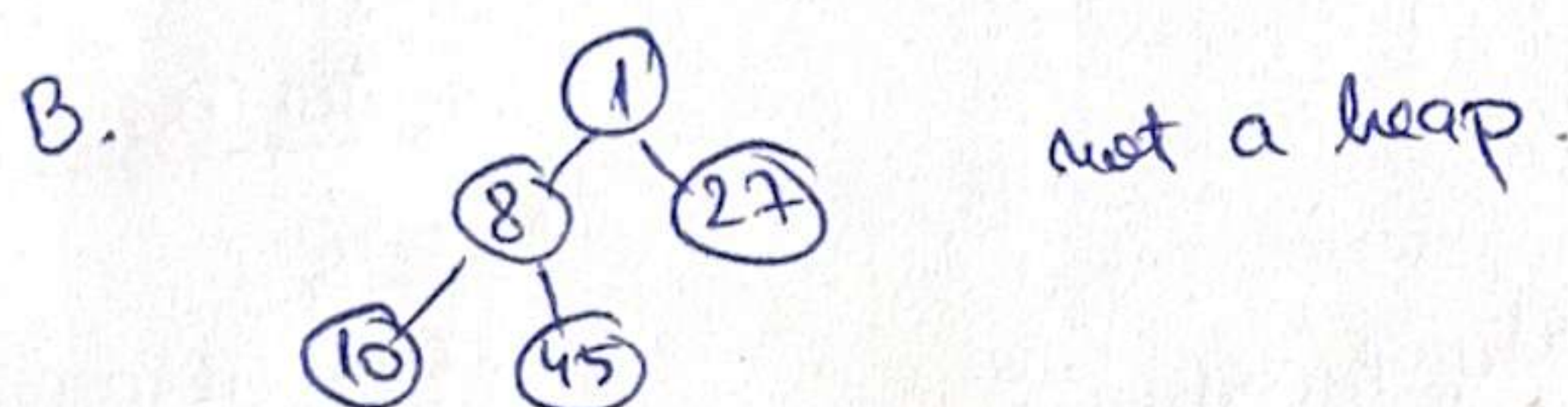
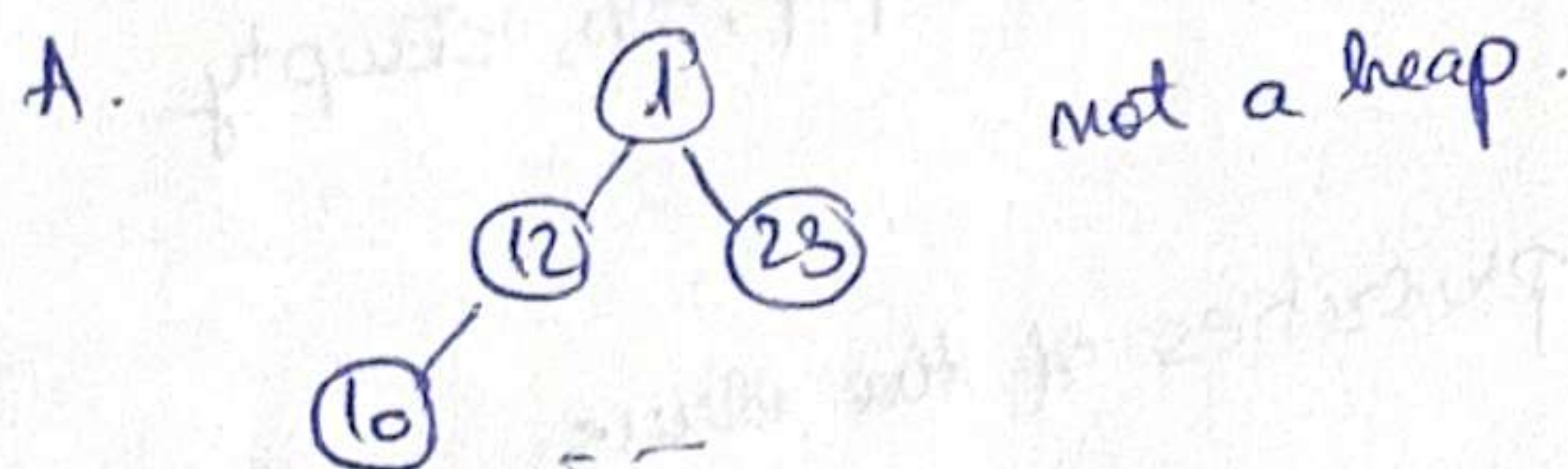
isEmpty  $\leftarrow$  isEmpty(q.pg)

```


1. Queue - add/delete

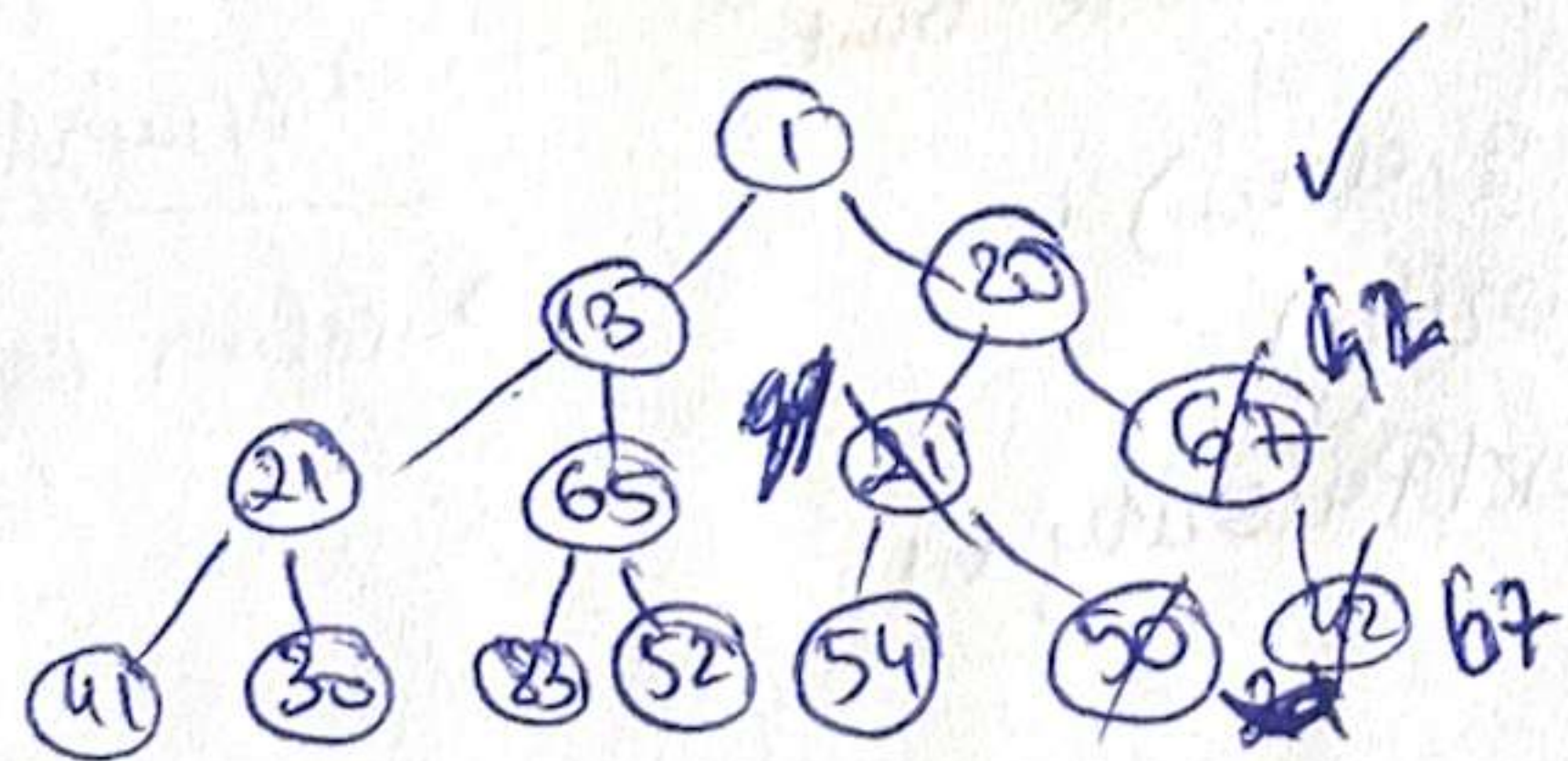
2. a) which is a binary heap?

- ↳ single + complete BT (max 2 children)
- ↳ every level filled
- ↳ prop + structure

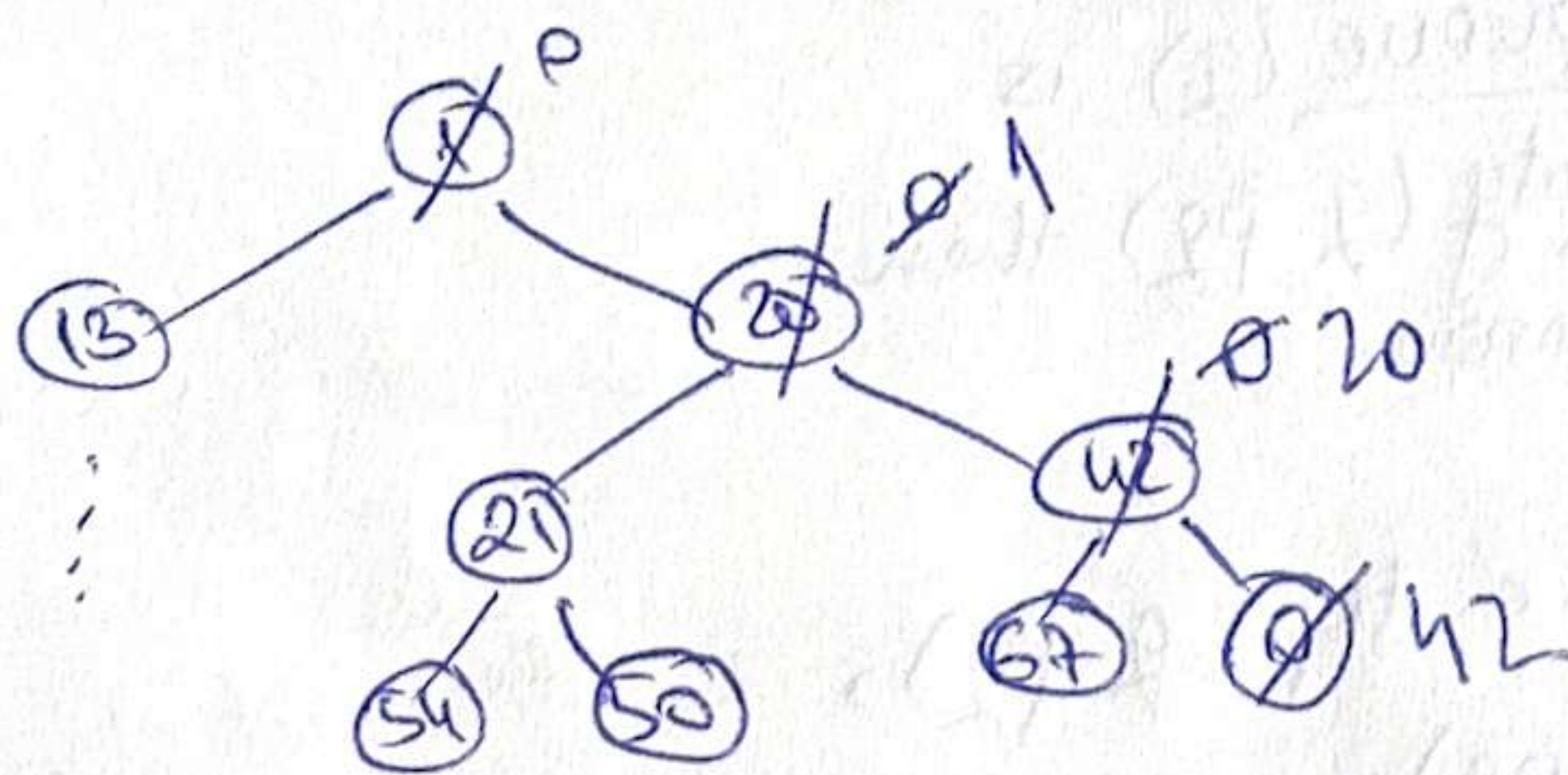


insert : 21, 50, 42, 0.

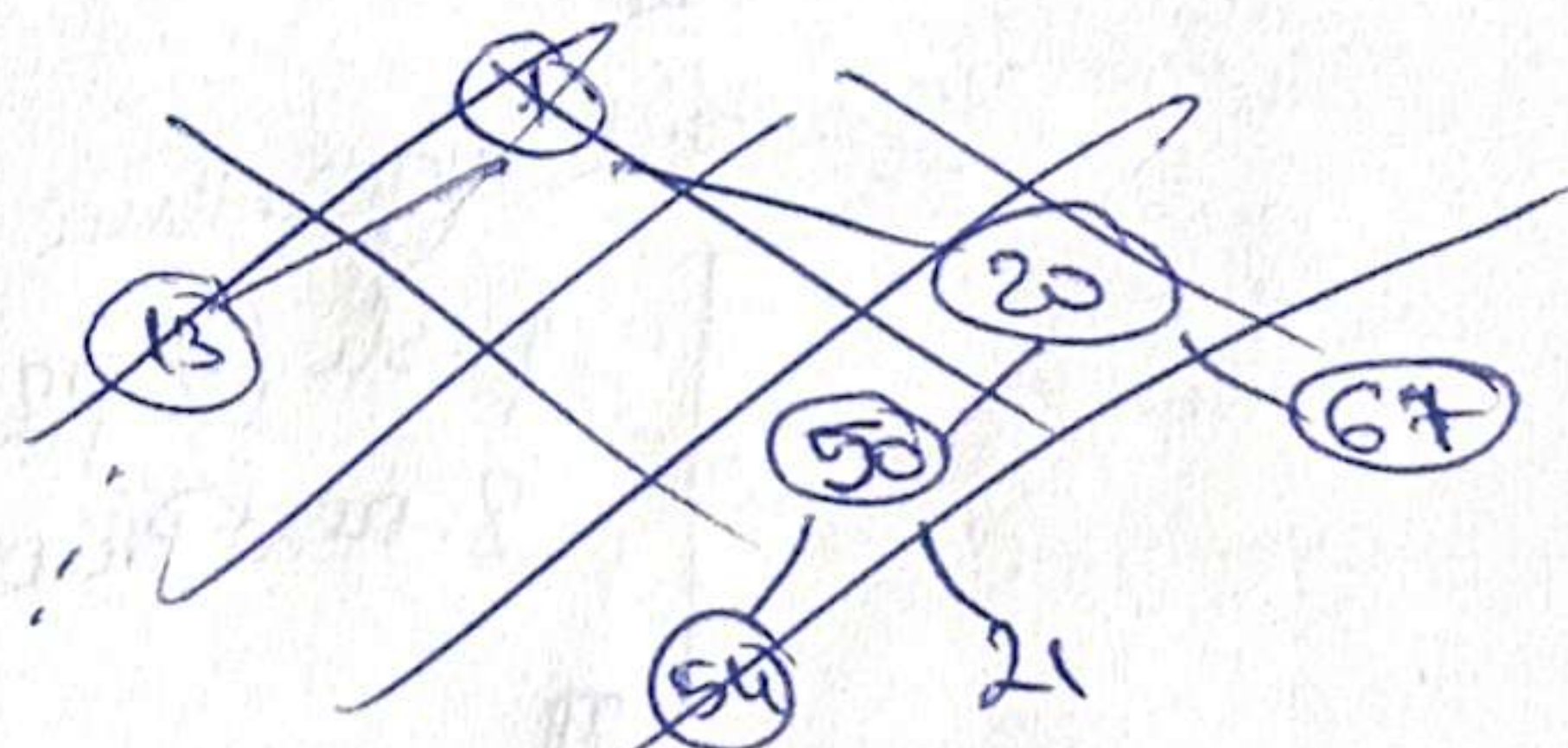
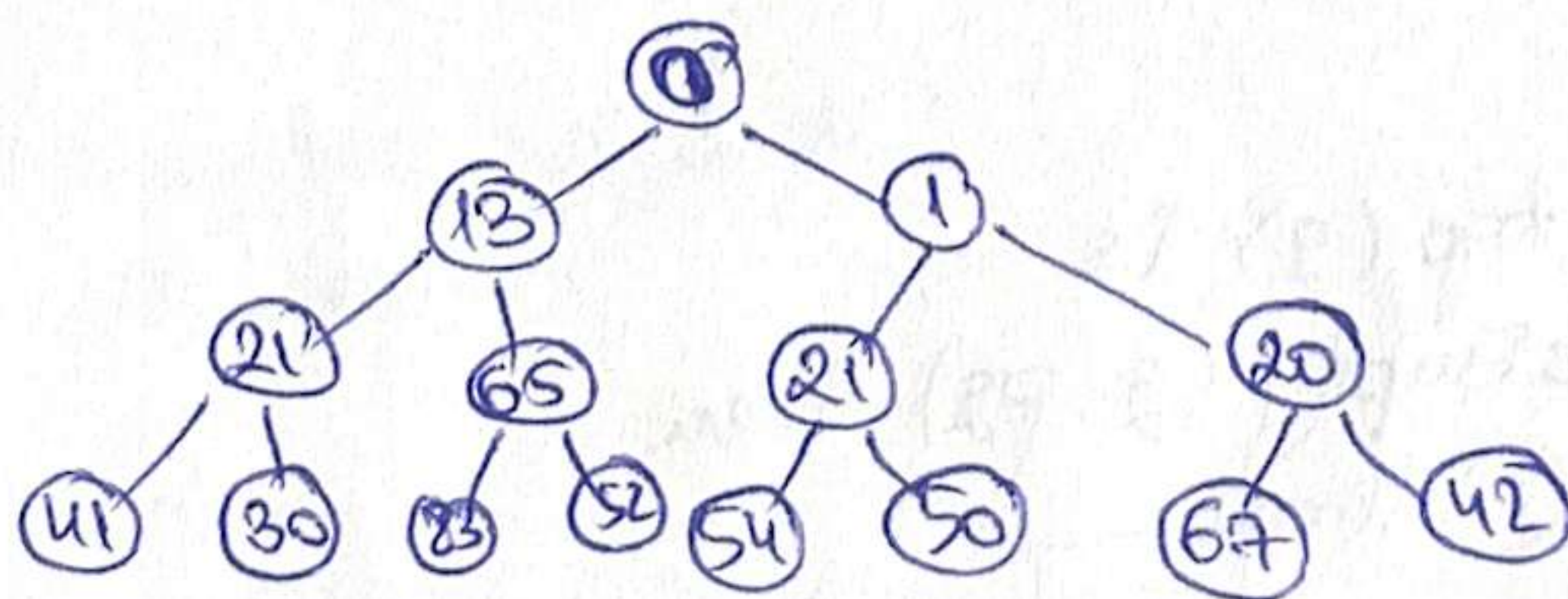
add 21
⇒
add 50
⇒



add 42
⇒



add 0
⇒

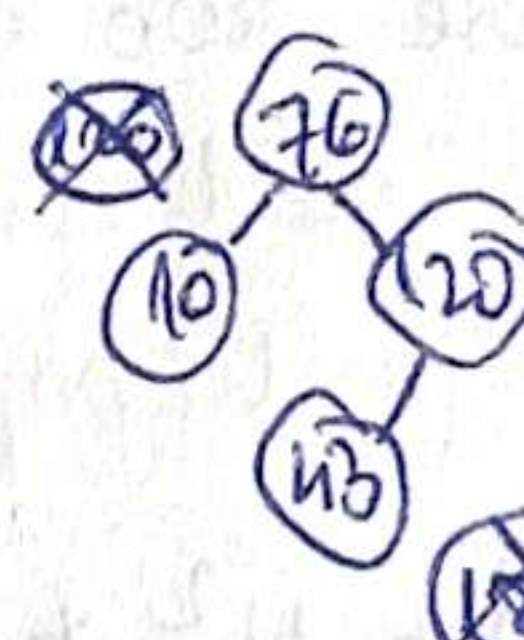
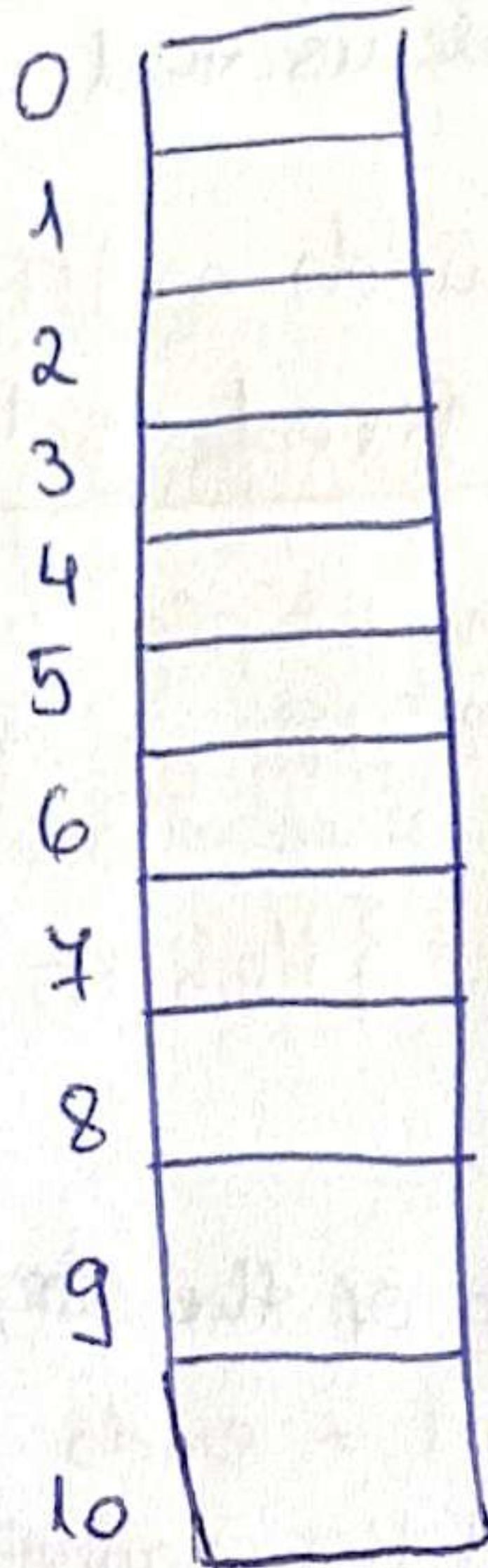


b) HT, $m=11$, exp. chaining, AVL tree.

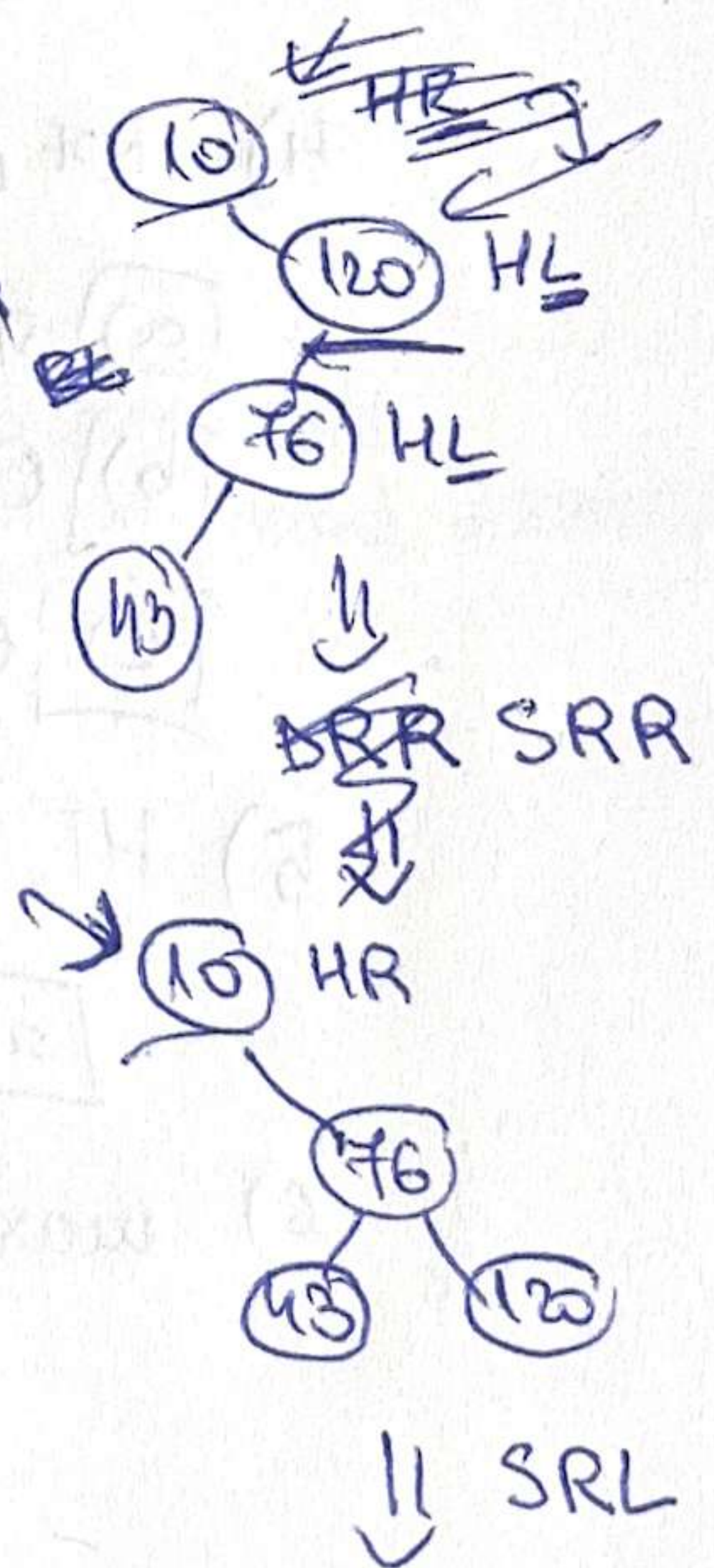
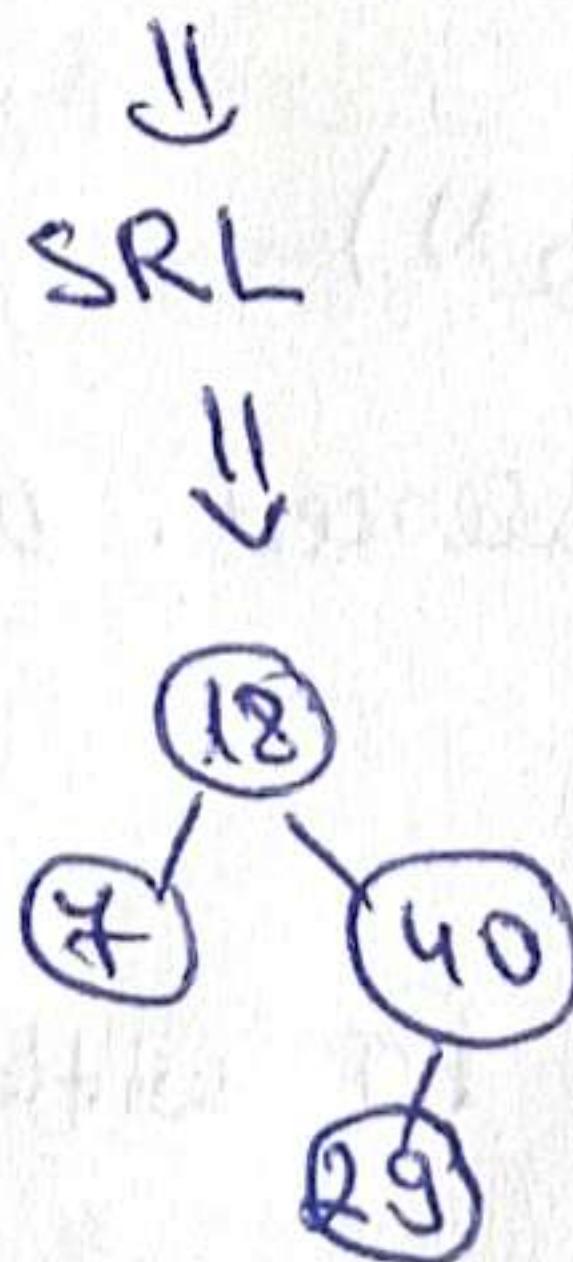
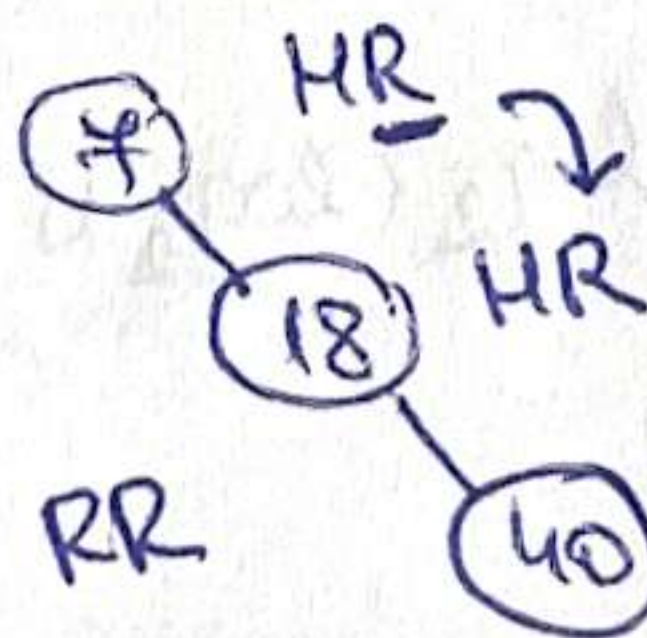
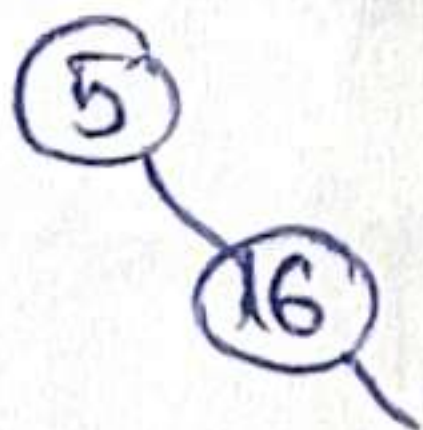
insert: 7, 10, 120, 5, 16, 18, 76, 40, 29, 43.

load factor: $\alpha = \frac{n}{m} = \frac{10}{11}$.

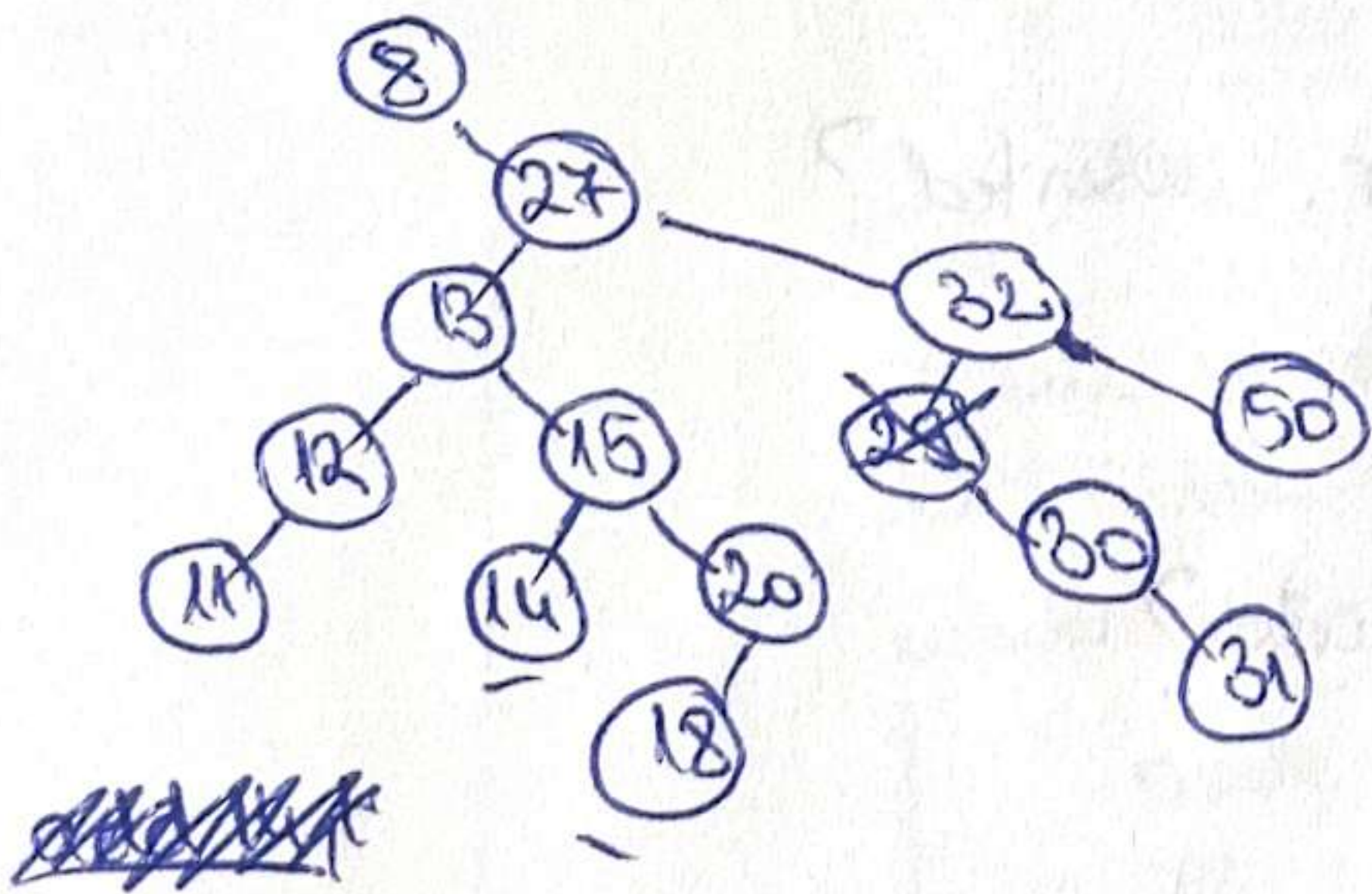
- $h(7) = 7 \% 11 = 7 \checkmark$
- $h(10) = 10 \checkmark$
- $h(120) = 10 \checkmark$
- $h(5) = 5 \checkmark$
- $h(16) = 5 \checkmark$
- $h(18) = 7 \checkmark$
- $h(76) = 10 \checkmark$
- $h(40) = 7 \checkmark$
- $h(29) = 7 \checkmark$
- $h(43) = 10$



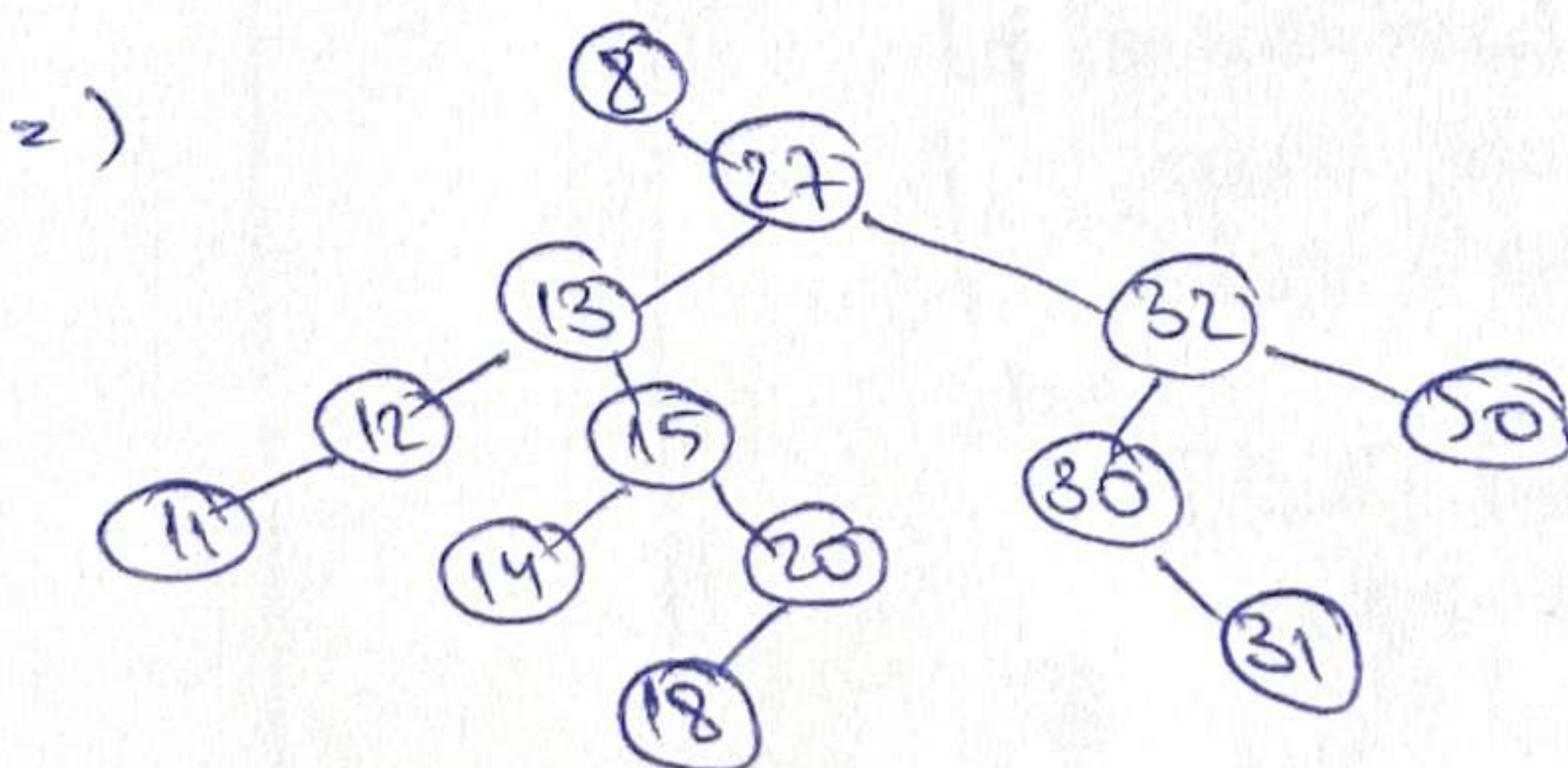
BST?



c) BST. (add 14, 18, 30, 31)



remove 29 (it only has right ch.)
 $\Rightarrow$ replace with 30.



If I had 2 children for 29, $\Rightarrow$ $\nearrow$ predecessor
 $\searrow$ successor

3)

1.) H bin. min. heap with > 1000 nr.
same as new...

2) main restriction of perfect hashing

[c] we need to know all elems. that will be inserted from the beginning

Perfect hashing: $\rightarrow$ builds a HT with no collisions for
a fixed set of keys.

$\rightarrow$ multiple hash functions

$\rightarrow$ HT of hash tables

$$H = \{H_{a,b}(x) = ((a \cdot x + b) \% p) \% m\}$$

3) FIFO queue vs LIFO stack

[c] queues use two ends of the str, stacks use one.

queue - push/pop at $\neq$ ends

stack - push/pop at the same end

$$4) N * (\log_2 N) + N^2 * (\log_2 (\log_2 N))$$

$$[a] \Omega(N * \log_2 N)$$

$$[b] O(N^3)$$

$$[c] \Theta(N^2 * \log_2 N)$$

5) HT, 100 pos, coalesced. max nr. inserted?

$$[a] 100$$

6) max depth of a BT with 16 nodes?

$$[d] 13$$

- 4) subalg. / merge K sorted SLL into one sorted SLL. (implem.)
 / receive an array of sorted SLL
 / return resulting list
 / $O(m \times \log_2 K)$, $m = \text{total nr. of elems. in the list. (final)}$
 / DS?
 / repres. for SLL
 / explain complexity $\rightarrow O(m \log K)$ because each of the m nodes is inserted/removed from a heap of size at most K .

DS: binary min heap / priority queue because we always need the smallest current head among the K lists.

SLLNode

info: Tcomp
 next: $\uparrow$ SLLNode

SLL

head: $\uparrow$ SLLNode
 rel: $\uparrow$ relation

subalgorithm merge(lists, K) is

init(heap)

for $i \leftarrow 0, K-1$ execute

if lists[i].head $\neq$ Nil then

add(heap, lists[i].head)

result.head $\leftarrow$ Nil

last $\leftarrow$ Nil

while not isEmpty(heap) execute

node $\leftarrow$ remove(heap)

if [node].next $\neq$ Nil then

add(heap, [node].next)

if result.head = Nil then

result.head $\leftarrow$ node

last $\leftarrow$ node

else

[last].next $\leftarrow$ node

last $\leftarrow$ node

if last $\neq$ Nil then [last].next $\leftarrow$ Nil

merge $\leftarrow$ result

lists[i] = sll

sll.head = lists[i].head

- 5) subalg - receives a bt
 - returns true/false $\rightarrow$ binary min. heap or not. (property & structure)
 / repres. BT (linked repres w dyn. alloc.)
 - implement subalg + complexity

Node:

info: TElem
 left: $\uparrow$ Node
 right: $\uparrow$ Node

BT:

root: $\uparrow$ Node

init(q)
 push(q, node)
 pop(q)
 isEmpty(q) } $\Theta(1)$

Queue $\rightarrow$ level-order-traversal (BFS) $\rightarrow$ once a missing child is encountered, every node visited afterwards must have no children.

function minHeap(bt)

if bt.root = Nil then
 minHeap $\leftarrow$ True

init(q)

push(q, bt.root)

foundGap $\leftarrow$ False

while not isEmpty(q) do execute

node $\leftarrow$ pop(q)

if node.left $\neq$ Nil then

if foundGap then

minHeap $\leftarrow$ False

if [node].info > [node.left].info then

minHeap $\leftarrow$ False

push(q, node.left)

else

foundGap $\leftarrow$ True

if node.right $\neq$ Nil then

if foundGap then

minHeap $\leftarrow$ False

if [node].info > [node.right].info then

minHeap $\leftarrow$ False

push(q, node.right)

else

foundGap $\leftarrow$ True

minHeap $\leftarrow$ True

BC: $\Theta(1)$

$\hookrightarrow$ empty tree or root only

WC: $\Theta(n)$

Total: $\Theta(n)$

each node: (n) nodes)

$\Theta(1)$ - dequeued once

$\Theta(1)$ - examined once

$\Theta(1)$ - enqueued at most once
 $\Rightarrow \Theta(n)$